

I'll read the system prompt carefully and design this purely as a product architecture exercise — no code, no components, just state machines, flows, and hard truths.

1. Frontend State Machine

Top-level application states:

```

APP
├── IDLE                (no active request, no active trip)
├── REQUESTING          (form being filled)
├── SUBMITTING          (POST in flight)
├── PENDING             (request exists, polling every 10s)
│   ├── [sub] PENDING_STALE (lost poll response, unclear if
still pending)
│   └── MATCHED          (trip created, polling every 30s)
│       ├── [sub] MATCHED_STALE (lost poll response during
trip)
│       └── COMPLETING    (POST /trips/:id/complete in flight)
├── COMPLETED          (terminal)
├── CANCELLED           (terminal – self-cancel or cascade)
│   └── [sub] REQUEUED    (was MATCHED, now back to PENDING – a
special PENDING entry)

```

Transient / loading states that must exist explicitly:

- **SUBMITTING** — distinct from REQUESTING; prevents double-submit
- **CANCELLING** — POST /cancel in flight; block re-tap
- **COMPLETING** — POST /complete in flight
- **POLLING_ERROR** — N consecutive poll failures (show degraded UI, keep retrying)
- **REHYDRATING** — app cold-start, checking if user has active state before showing anything

REHYDRATING is the most commonly forgotten state. If a user refreshes mid-PENDING, the app must re-derive state from the API before rendering anything. Without it, you'll flash IDLE incorrectly.

2. Screen Flow Map

Driven by state, not routes:

```

REHYDRATING
|
|-[has PENDING request]----->
PENDING_SCREEN
|
|-[has MATCHED trip]----->
TRIP_SCREEN

```

└[no active state]—————→
HOME_SCREEN
└[API error]—————→
ERROR_SCREEN (retry)

HOME_SCREEN
└[tap "Find Ride"]—————→
REQUEST_FORM_SCREEN

REQUEST_FORM_SCREEN
└[submit]—SUBMITTING—[success]—————→
PENDING_SCREEN
[duplicate PENDING error]→
HOME_SCREEN (show toast)
[validation error]—————→
REQUEST_FORM_SCREEN

PENDING_SCREEN (polling every 10s)
└[poll → status=MATCHED]—————→
TRIP_SCREEN
└[poll → status=CANCELLED (auto-cancel)]—————→
HOME_SCREEN (show "not matched" toast)
└[user taps cancel]—CANCELLING—[success]—————→
HOME_SCREEN
└[N poll failures]—————→
PENDING_SCREEN with degraded banner

TRIP_SCREEN (polling every 30s)
└[poll → trip status=CANCELLED (co-rider cancelled)]►
REQUEUED_SCREEN
└[user taps "Complete Ride"]—COMPLETING—[success]→
COMPLETED_SCREEN
└[user taps cancel]—CANCELLING—[success]—————→
HOME_SCREEN
└[N poll failures]—————→
TRIP_SCREEN with degraded banner

REQUEUED_SCREEN (brief interstitial, then auto-transition)
└[after dismiss or auto]—————→
PENDING_SCREEN (new PENDING state)

COMPLETED_SCREEN (terminal)
└[tap "Done"]—————→
HOME_SCREEN

3. Critical User Flows

Flow A: Normal match and complete

HOME → fill form → SUBMITTING → PENDING (polling)
→ poll detects MATCHED → TRIP_SCREEN
→ tap Complete → COMPLETING → COMPLETED → HOME

Flow B: User cancels while PENDING

PENDING_SCREEN → tap Cancel → CANCELLING
→ POST /cancel → response: {status: CANCELLED,
cancelledTripId: null}
→ HOME_SCREEN

Flow C: User cancels while MATCHED (cascade initiator)

TRIP_SCREEN → tap Cancel → CANCELLING
→ POST /cancel → response: {status: CANCELLED,
cancelledTripId: "xyz", requeuedUserIds: [...]}
→ HOME_SCREEN (show "Your trip was cancelled" toast)

Flow D: Co-rider cancels (cascade victim) — the hard one

TRIP_SCREEN (polling every 30s)
→ poll GET /trips/:id → trip.status = CANCELLED
 OR poll GET /ride-requests → finds a PENDING request the
user didn't explicitly cancel
→ transition to REQUEUED_SCREEN
→ show "A co-rider cancelled. You've been re-queued."
→ auto-transition to PENDING_SCREEN (resume 10s polling)
The detection mechanism here is subtle: you can't rely on the trip poll alone. If the user's ride request reverts to PENDING, the trip endpoint may return CANCELLED but the user needs to know they're back in queue, not just dumped. The app must check both signals.

Flow E: App cold-start with active state

App opens → REHYDRATING
→ GET /ride-requests?status=PENDING → found one
→ jump directly to PENDING_SCREEN (skip HOME entirely)

Flow F: Auto-cancel after 5 cycles

PENDING_SCREEN (polling)
→ poll → status=CANCELLED (pendingCycles reached 5, user did nothing)
→ HOME_SCREEN with toast: "No match found. Request expired."

4. Edge Cases Frontend Must Handle

1. The phantom PENDING on rehydration User was MATCHED, co-rider cancelled, their request reverted to PENDING. User closes and reopens the app. The rehydration check finds a

PENDING request — but this is a *requeued* PENDING, not a fresh one. The frontend can't tell the difference from the API response alone since `pendingCycles` is preserved. You need to either show a generic "you're in queue" screen without assuming it's fresh, or add a flag to the response.

2. Race between cancel and match User taps Cancel at the exact moment matching runs and transitions them to MATCHED. The cancel POST hits a MATCHED request instead of PENDING. Your backend handles this (cascade cancel), but the frontend will receive `{was_matched: true, cancelledTripId: "..."}` when it expected `{was_matched: false}`. The UI must handle both cancel response shapes from the same user action.

3. Poll returns stale data Network hiccup returns a cached or delayed response with `status=PENDING` when the user is actually MATCHED. Frontend should not regress state (never go MATCHED → PENDING based on a single poll). Always treat state transitions as monotonically forward, not bidirectional — except for the explicitly defined MATCHED → PENDING requeue path.

4. Complete button on an already-cancelled trip Co-rider cancels the trip. User's poll hasn't fired yet (up to 30s window). User taps "Complete Ride." POST `/trips/:id/complete` will return 400 TRIP_NOT_COMPLETABLE. Frontend must handle this gracefully and immediately re-poll to discover the cancellation rather than just showing a generic error.

5. Double-submit on slow connections SUBMITTING state must disable the submit button. But also: if the POST `/ride-requests` succeeds but the response is lost (network drop), user retries — backend throws DUPLICATE_PENDING_REQUEST (409). Frontend must interpret 409 on create as "you already have one, go poll it" and transition to PENDING, not show an error.

6. Session continuity across tabs/devices No auth system means `user_id` is passed via header. If two tabs are open, both will poll independently and could show conflicting states. This is a known constraint given no WebSockets, but the UI should avoid letting users submit a second request from a stale tab (the 409 handling above covers recovery).

5. Flawed Assumptions and Backend Risks

Risk 1: The 30-second co-rider cancellation gap is a real UX problem. Your polling interval for trips is 30 seconds. A co-rider can cancel, and the victim user's UI will show an active trip with a "Complete" button for up to 30 seconds after the trip is dead. If they tap Complete in that window, they get an error. This isn't a bug you can fix cleanly on the frontend — it's a fundamental polling latency issue. The mitigation is to treat any unexpected error on trip actions as a trigger to immediately re-poll, not just show an error message.

Risk 2: Requeue detection is ambiguous. When a co-rider cancels and the victim is re-queued, your backend sets their ride request status back to PENDING. The frontend detects this via the trip poll (trip=CANCELLED) or the ride-request poll. But both polls run on different cadences (30s trip, 10s ride-request). The frontend needs a clear decision on which signal is authoritative. My recommendation: when trip status=CANCELLED and the user didn't initiate the cancel, immediately switch to polling ride-requests at 10s cadence. Don't wait for the 30s trip poll to cycle again.

Risk 3: No explicit "was_cancelled_by_corider" field. Your API spec says the user detects co-rider cancellation by finding a PENDING request they didn't explicitly cancel. That's fragile. The

only way to distinguish "I just submitted a request" from "I was requeued" is `pendingCycles > 0`. That's an implicit contract. If `pendingCycles` is 0 after requeue (which it shouldn't be per your spec, but bugs happen), the frontend cannot distinguish these cases. Consider adding an explicit `requeued: true` flag on the ride request, or a `cancelledByCorider` field on the trip response.

Risk 4: The IDLE state doesn't actually exist cleanly. A user can be in IDLE with a CANCELLED historical ride request in the DB. IDLE means "no active request" — but your GET / ride-requests returns all requests. The frontend must filter to only the most recent PENDING or MATCHED record and ignore historical CANCELLED ones. If your endpoint doesn't return the requests in a clearly ordered way, this filtering becomes error-prone.

Risk 5: Auto-cancel is silent. When a request is auto-cancelled after 5 cycles, the user has no push notification (no WebSockets). They'll only find out on the next poll. If they're not actively looking at the app, they may never know. This is fine for a demo, but the frontend should clearly show "Your request expired" on next open rather than silently dropping to IDLE. The `pendingCycles` field can be surfaced on the PENDING screen to give users feedback ("matched in X out of 5 attempts" or similar).

6. Missing States

Missing: CANCELLATION_CONFLICT When the cancel POST returns an unexpected response (e.g., trip already cancelled by someone else mid-request), the frontend has no defined state for "cancel was attempted but the entity was already in a terminal state." Right now this is lumped into generic error handling. It should be an explicit resolution path: detect the conflict, re-poll, and settle into the correct terminal state.

Missing: REQUEST_EXPIRED Auto-cancel should have its own distinct terminal display state separate from user-initiated CANCELLED. Both end at the same API status value (CANCELLED) but they represent different user experiences and need different messaging. The frontend will conflate them without an explicit state.

Missing: TRIP_STALE (distinct from MATCHED_STALE) When the app is polling and the server is slow, there's a window where the trip data is potentially 30 seconds out of date and the user is about to take an action (Complete, Cancel) on it. A TRIP_STALE state — triggered after N failed polls — should visually warn the user before allowing actions that could fail or produce confusing errors.

Missing: FORM_PREFILLED After a co-rider cancels and the user is requeued, they're back to PENDING with their *original* request. They didn't re-fill the form. If they want to cancel and start fresh with different coordinates, the form needs to know to pre-fill with the existing pending request's data, not show a blank form. This is a state the frontend needs to track — "is this a fresh request or a requeue-derived one."